Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Nombre del estudiante(s)	Israel Rodríguez Isabel Morocho Fabricio Ruiz Mateo Rojas
Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	1
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001
Título de la Práctica	Análisis Colaborativo de las Arquitecturas Principales en un Sistema Distribuido
Nombre del Docente	José O. Guamán Q.
Fecha	29 de Septiembre de 2026 02 de Octubre de 2026
Horario	Martes 10h30 – 11h30 VIernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Analizar de manera colaborativa las arquitecturas fundamentales de los sistemas distribuidos (cliente-servidor, peer-to-peer, multicapa y microservicios), identificando sus características, ventajas, desventajas y casos de uso, mediante trabajo en equipo y el diseño de un caso de estudio.

- Identificar las características estructurales y funcionales de las arquitecturas cliente-servidor, peer-to-peer, multicapa (n-tier), y microservicios en sistemas distribuidos.
- Comparar las ventajas, desventajas, escalabilidad y tolerancia a fallos de cada arquitectura mediante el llenado colaborativo de una matriz de análisis.
- Diseñar el diagrama de un caso de estudio propuesto aplicando la arquitectura distribuida más adecuada, justificando técnicamente la elección del equipo.

3. Materiales, Reactivos:

Diapositivas, y enunciados del problema:

- Hardware:
  - Computadora personal con conexión a internet.
- Software:
  - Procesador de texto.

4. Equipos y Herramientas

Computadora personal con conexión a internet

Guía de práctica.

Plantilla de matriz comparativa

5. Procedimiento / Metodología

Fase 0 (10 min). Formación de equipos de 4 (asignación del docente).

Fase 1 – Investigación (50 min). Cada integrante investiga su arquitectura y completa su sección de la matriz (estructura, ventajas, desventajas, escalabilidad, ejemplos). El equipo elabora el diagrama de su arquitectura en draw.io.

Fase 2 – Socialización (30 min). Cada equipo expone 5 min. El resto completa la matriz comparativa con la información expuesta (una sola matriz, grupal).

Fase 3 – Caso de estudio (55 min). Cada equipo selecciona la arquitectura más adecuada para el enunciado, la justifica con mínimo 3 argumentos técnicos, elabora el diagrama de la solución e identifica dónde se aplican transparencia, escalabilidad y tolerancia a fallos.

Fase 4 – Reporte y entrega (25 min). Cada estudiante redacta sus conclusiones individuales; el equipo compila la matriz final y los diagramas. Entrega en la plataforma indicada.

Criterio	Cliente - Servidor
Estructura	- **Cliente:** Es el dispositivo o software con el que interactúa el usuario, es responsable de capturar las entradas del usuario, enviar las peticiones al servidor y almacenar información temporal. - **Servidor**: Es el sistema centralizado que gestiona los recursos, la lógica que se ejecuta, el estado de la aplicación y se encarga de procesar las solicitudes de los múltiples clientes conectados. - **El Canal y Protocolos de Comunicación:** Es la infraestructura de red y los componentes encargados de controlar la comunicación, encargados de regular el intercambio de mensajes, solicitudes y actualizaciones entre el cliente y el servidor.
Ventajas	Esta arquitectura nos proporciona un control administrativo y gestión centralizada al tener la lógica de negocio y la base de datos en el servidor. Debido a esta distribución podemos tener un mayor control en aspectos de seguridad. Resulta más sencilla de diseñar e implementar en comparación con arquitecturas como Peer-to-Peer (P2P), ya que no requiere gestionar el descubrimiento de pares.
Desventajas	Al concentrar los servicios y la lógica de negocio en un servidor central tenemos dependencia centralizada y punto único de fallo, ya que si este servidor cae o es inaccesible, todos los clientes pierden la capacidad de consumir el servicio o recuperar la información.
Escalabilidad	Tenemos dos enfoques principales, la escalabilidad vertical, que consiste en incrementar los recursos de hardware del servidor central y la escalabilidad horizontal que consiste en incorporar varios servidores que trabajan de manera conjunta para distribuir la carga de trabajo.
Ejemplos reales	1. **Navegación web:** El cliente es el navegador que utiliza el usuario, este recibe las instrucciones y presenta el contenido. El servidor web almacena las páginas y recursos necesarios, procesa las solicitudes y envía la información que es solicitada por el navegador. 2. **Juegos en Línea:** En los juegos multijugador el cliente es el juego instalado en nuestra PC y es el encargado de recibir los inputs del usuario y mostrar los gráficos. El servidor administra la lógica de lo que sucede en el juego, el estado de cada jugador y sincroniza esta información para todos.

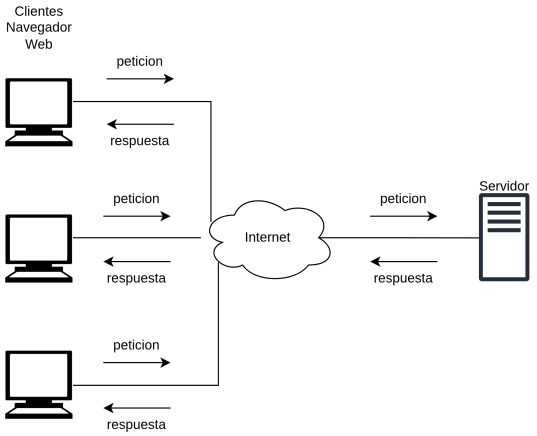


Figura 1: Diagrama de la arquitectura Cliente - Servidor

Criterio	Peer-to-Peer
Estructura	En esta arquitectura cada nodo participante (peer) opera de manera descentralizada y puede actuar tanto como consumidor de recursos (cliente) como proveedor de recursos (servidor). Según la forma en que los nodos se organizan y descubren recursos, las redes P2P pueden clasificarse en dos categorías: 1. **Estructurada:** Utilizan algoritmos para garantizar una búsqueda eficiente y determinista de los recursos. 2. **No estructurada:** Los nodos se conectan de forma aleatoria, y utilizan mecanismos de consulta entre peers para localizar los recursos.
Ventajas	Como ventajas, se debe indicar que esta arquitectura es tolerante a fallos ya que al no existir un servidor central, todos los nodos pueden asumir la distribución de información. Asimismo, se reduce o elimina la necesidad de mantener infraestructura de servidores centralizados, ya que los propios peers pueden aportar recursos a la red.
Desventajas	Como desventajas, se debe mencionar que la falta de una autoridad central dificulta la autenticación, el control y la regulación de los nodos, lo que puede facilitar la distribución de contenido o software malicioso. Además su administración y coordinación son más complejas, debido a que los nodos pueden unirse y abandonar la red con frecuencia y de manera impredecible.
Escalabilidad	En cuanto a la escalabilidad, en una red P2P cada nuevo nodo que ingresa para consumir recursos puede aportar simultáneamente sus propios recursos de subida y procesamiento a la red. Sin embargo, la escalabilidad puede verse afectada dependiendo de los mecanismos utilizados para descubrir y coordinar los nodos.
Ejemplos reales	1. **BitTorrent:** Mecanismo que distribuye archivos pesados mediante pequeños fragmentos (chunks), permitiendo así que los usuarios descarguen partes de un archivo desde múltiples nodos simultáneamente y, a su vez, compartan los fragmentos que ya poseen.

	2. **Windows Delivery Optimization:** Mecanismo utilizado por Windows para distribuir actualizaciones, mediante el cual los equipos pueden descargar contenido desde servidores de Microsoft y, dependiendo de la configuración, obtenerlo o compartirlo con otros dispositivos.

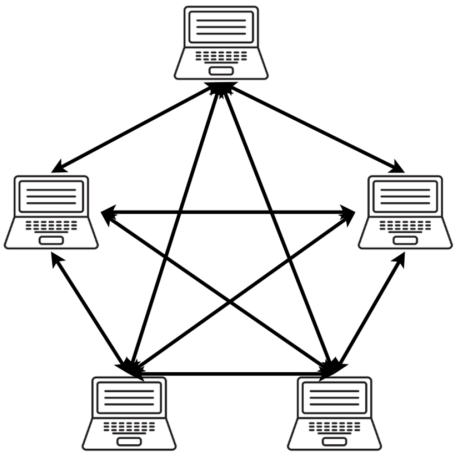


Figura 2: Diagrama de la arquitectura Peer-to-Peer

Criterio	Multicapa
Estructura	Esta arquitectura surgió tradicionalmente con tres capas (Presentación, Negocio y Datos), pero es un modelo extensible. En sistemas complejos, estas capas primarias se subdividen en subcapas (como persistencia, integración, clientes ligeros/pesados o servicios API) para lograr un mayor nivel de desacoplamiento. Dentro de las capas principales se encuentran las siguientes: - **Capa de presentación:** Más conocida como interfaz gráfica, es la encargada de presentar los resultados al usuario, y recolecta la información que genera el usuario al sistema. - **Capa de negocio o lógica de la aplicación:** Da la funcionalidad de la aplicación, y realiza todas las operaciones, procesa los datos, es decir toma las peticiones del usuario y decide qué hacer con ellas - **Capa de datos o persistencia:** Almacena la información de la aplicación, asegura la información de manera controlada y segura, en esta capa se conecta con la base de datos o sistemas externos
Ventajas	- **Más utilizada y versátil:** Es una de las arquitecturas más usadas para servicios web, desde plataformas de comercio electrónico hasta aplicaciones sociales - **Fácilmente escalable:** Se puede escalar horizontalmente añadiendo más servidores. permitiendo al sistema manejar grandes cantidades de datos sin problema
Desventajas	- La principal desventaja es la pérdida de rendimiento e incremento de la latencia debido al tiempo que se demoran los datos en viajar y comunicarse con las diferentes capas.

	- Al dividir el sistema en mucho niveles se requiere escribir más código repetitivo y el diseño inicial se vuelve más complejo de planificar - Esta arquitectura exige más consumo de recursos como memoria, y capacidad de procesamiento en los servidores, lo cual encarece a la infraestructura requiriendo actualizaciones de hardware más seguido
Escalabilidad	En cuanto a la escalabilidad permite que cada capa escale de manera independiente según requiera el sistema. - En la capa de presentación se añaden más servidores web para atender a una gran cantidad de usuarios - En la capa de negocio se dividen los procesos en distintos nodos de procesamiento, evitando cuellos de botella - En la capa de datos se incrementan técnicas de replicación, particionamiento o bases de datos distribuidas para resistir gran cantidad de información
Ejemplos reales	**1. Aplicación web de un carrito de compras en sitios de comercio electrónico** - **Capa de cliente:** (Navegador) presenta la interfaz gráfica con la que el usuario interactúa - **Capa de presentación:** Recibe las peticiones del usuario (funcionalidades), además coordina las respuestas con las demás capas y llama a servicios externos para obtener respuestas rápidas. - **Capa de aplicación:** Maneja las reglas del negocio y actúa como puente de integración para que la presentación y los datos se comuniquen sin dificultad - **Capa datos:** Guarda y gestiona la información de manera independiente, mejorando la escalabilidad y rendimiento del sistema **2. Sistema de banca en línea** - **Capa de presentación:** Representa la aplicación que el usuario descarga en su teléfono móvil, esta capa captura las solicitudes del usuario y renderiza interfaces - **Capa de negocio:** El servidor central recibe las peticiones, ejecuta las reglas del negocio (verifica si tiene saldo, el estado de la cuenta y transacciones). - **Capa de datos:** Registra de manera inmutable el nuevo saldo cuando la capa de negocio haya dado el visto bueno

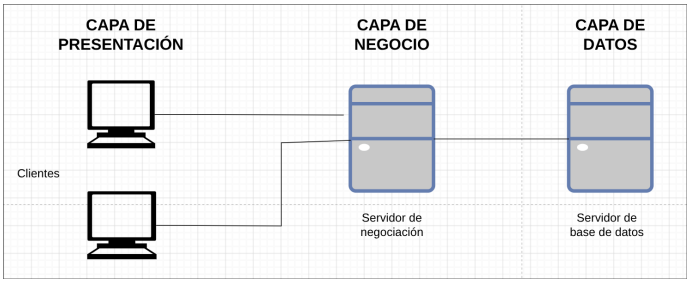


Figura 3: Diagrama de la arquitectura Multicapa

Criterio	Microservicios
Estructura	La estructura de los microservicios va desde una aplicación dividida en pequeños servicios independientes que están acoplados, organizados por las capacidades del negocio, este enfoque permite implementar y escalar cada servicio individual de forma independiente, este enfoque permite implementar y escalar cada servicio individual de forma independiente, así como la entrega rápida y frecuente de aplicaciones grandes y complejas.
Ventajas	El despliegue y actualización independiente, tolera a fallos que están aislados, si un servicio falla o se cae, el resto sigue funcionando, lo que permite usar diferentes lenguajes y bases de datos según su necesidad, esto facilita la actualización del código y acelera el tiempo de salida al mercado de nuevas funciones, permite que los equipos pequeños especializados se centren en el desarrollo, la implementación y el mantenimiento de un servicio en particular sin verse agobiados por las complejidades de todo el sistema.
Desventajas	Se necesita una alta complejidad operativa lo cual la latencia de red por la comunicación entre servicios tiene dificultad para mantener la consistencia de datos en transacciones distribuidas, puede resultar difícil depurar una aplicación que contiene varios microservicios, cada uno con su propio conjunto de registros, un único proceso empresarial puede ejecutarse en varias máquinas simultáneamente, lo que agrava la complejidad.
Escalabilidad	Es altamente granular y eficaz que permite escalar de forma independiente, horizontal únicamente al microservicio que presenta una alta demanda, sin tener que escalar todo el sistema, gracias al funcionamiento independiente de cada servicio dentro de la arquitectura de microservicios, los equipos de DevOps introducen fácilmente nuevos componentes sin provocar ningún tiempo de inactividad.
Ejemplos reales	• Uber utiliza una arquitectura basada en microservicios para poder manejar de forma independiente las diferentes funciones de su plataforma. Por ejemplo, puede contar con servicios separados para el registro de usuarios y conductores, la ubicación mediante GPS, la búsqueda de conductores cercanos, la solicitud de viajes, el cálculo de tarifas, los pagos, las calificaciones y las notificaciones. • Spotify también utiliza microservicios para dividir las distintas funciones de su plataforma de música. Puede tener servicios independientes para gestionar usuarios, canciones, artistas, álbumes, listas de reproducción, búsquedas, recomendaciones, historial de reproducción y suscripciones. Por ejemplo, cuando una persona escucha música, un servicio se encarga de reproducir la canción, mientras otro registra el historial y otro analiza sus gustos para ofrecer nuevas recomendaciones.

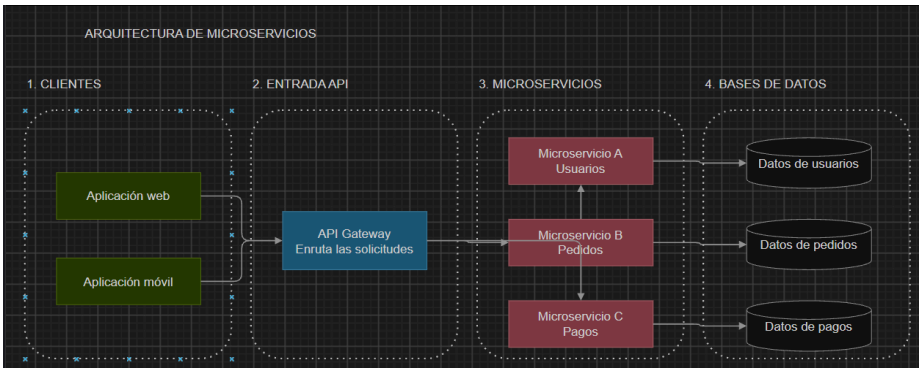


Figura 4: Diagrama de la arquitectura Microservicios

Fase 5: Socialización y matriz comparativa

1. Cada equipo expone su arquitectura durante 5 minutos (proyector)

2. Durante las exposiciones, todos los equipos llenan la matriz comparativa con la información de las cuatro arquitecturas:

Grupo	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
1	Estructura	Interviene el usuario y la aplicación.	Interconexión de distintos nodos, actuando tanto como clientes como servidores.	Varias capas: capa de presentación, capa del negocio, capa intermedia entre reglas de negocio uy base de datos y capa de datos	Escribir servicios para que estén juntos cada microservicio actúa de manera diferente, cada servicio tiene su propia responsabilidad.
	Ventajas	Nos permite organización y mantenimiento Facilita el control y acceso.	Escalabilidad muy alta, propiedad compartida.	Organización jerárquica, nos ofrece gran seguridad	Se pueden modificar por separado no se necesita interrumpir la acción de código, se encargan independientemente.
	Desventajas	Único servidor centralizado, lo que implica riesgos.	Alta volatilidad de nodos.	Este sistema tiene muchos nodos y dificulta la comunicación entre ellos, se dificulta el tiempo de respuesta	Son servicios separados y hacen más complicados la utilidad de la aplicación puede causar demora, se puede tomar más tiempo en si hay un error
	Escalabilidad	Implementación por aumento de recursos.	Ilimitada y global.	Cada capa escala de manera independiente	Se puede aumentar la capacidad de cada servicio añadiendo más distancia con el fin de aprovechar el hardware de mejor manera.
	Ejemplos reales	Sistema Académico por ejemplo el SGA.	Skype, BitTorrent.	Sistema bancario	Soundcloud, Netflix
2	Estructura	Un cliente que hace peticiones y un servidor centralizado que las procesa.	Se organiza en cuatro modelos: Estructurada, No estructurada, Híbrida, Descentralizada.	Capa independientes con sus responsabilidades separadas	Es la arquitectura que complementa el divide y vencerás se trata de generar acciones específicas y aisladas que se comunican entre sí, contiene el gateway y los orquestados.
	Ventajas	Arquitectura centralizada.	No depende de un único servidor, es tolerante a fallos, pueden compartir recursos entre sí.	Aislamiento entre capas, se puede hacer simulaciones entre capas	Contiene una libertad de elección de varias tecnologías, la facilidad de mantenimientos con servicio externos o locales.
	Desventajas	Único punto de fallo.	Difícil de encontrar el nodo que falló. Los nodos no tienen el mismo rendimiento.	Cuando hay muchas peticiones afecta el rendimiento y la latencia. Y riesgo de bloqueo tecnológico	El mantenimiento es complejo por la gestión de múltiples servicios, la consistencia de datos está de manera independiente.
	Escalabilidad	Se puede aumentar la capacidad agregando hardware.	Mientras más nodos es mejor.	Separación de responsabilidades en cada capa	Esta arquitectura puede utilizar un mejor lenguaje o tecnología para cada servicio sin preocuparse de la compatibilidad.

	Ejemplos reales	Tiendas online, donde el cliente hace una petición y el servidor devuelve lo necesario.	Criptomonedas, BitTorrent.	ConStore, librería de Java, Heath Watcher, sistema web de gestión de salud	Spotify, JP morgan
3	**Estructura**	Estructura centralizada con clientes y un servidor.	Todos los nodos son iguales, pueden actuar como cliente o como servidor.	Capa de presentación, lógica de negocio y acceso de datos	Cada microservicios esta compuesta por una api gateway
	Ventajas	Seguridad, acceso a los mismos servicios y fácil mantenimiento al estar centralizado.	Compartir información.	Cada capa se maneja individualmente o independiente	Cada microservicio puede ser actualizado si afectar a los otro s microservicios, el nivel de datos es más fácil de manejar
	Desventajas	Si el servidor falla, todos los clientes pierden los servicios.	Pueden ser bloqueados de acuerdo a las redes.	Para sistemas pequeños se usa otras arquitecturas	Puede haber una inconsistencia se puede hacer un bucle que acaba microservicio contiene una base datos en vez de unificarla
	Escalabilidad	Vertical, aumentar recursos. Horizontal, agregar más nodos.	Escalabilidad altra, pueden unirse más nodos.	Latencia entre capas	Tiene escalabilidad horizontal y vertical puede subir y no afectar a los demás microservicios
	Ejemplos reales	Aplicación de banco donde solicitamos peticiones sobre nuestros datos financieros.	Bases de datos.	Banco que se maneja por capas	Uber
4	**Estructura**	Arquitectura centralizada con clientes que solicitan a un servidor.	Cada dispositivo puede actuar como cliente o como servidor, es descentralizado	Funcionalidades diferentes, capa de presentación (interfaces), capa de negocio, capa de almacenamiento (base de datos)	Enrutamiento segmentado, frontend con base de datos propia
	Ventajas	Desacoplado, conecta múltiples clientes simultáneamente.	No requiere de un administrador central, es tolerante a fallos, comunicación directa.	Tiene tareas específicas, pueden funcionar individualmente	Independencia y libertad de lenguaje de programación por mIdielo
	Desventajas	Congestión con muchas peticiones.	Alta complejidad, riesgo de saturación.	Genera retrasos y latencia para el servicio	Mayor filtración de datos
	Escalabilidad	Aumento de Hardware.	Infinita, cada nodo que ingresa, es un recurso adicional.	Alata, se puede hacer cambios y funcionalidades	Inversión de recursos específicos
	Ejemplos reales	Sistema académico donde el cliente solicita y el servidor responde.	Bitcoin (en transacciones)	Aplicaciones bancarias, y sistema de matrículas	Billeteras digitales
6	**Estructura**	Modelo centralizado donde los clientes realizan solicitudes y un servidor central responde.	Nodos simultáneamente cliente y servidores	Capa de presentación, negocio y datos	Servicios pequeños desacoplados comunicados via red
	Ventajas	Arquitectura centralizada.	Escalabilidad alta.	Cada capa realiza su tarea específica	Arquitectura ágil
	Desventajas	Punto central de fallo que deja sin servicio a los clientes.	Fallos de seguridad	Alta dependencia	Arquitectura compleja al presentar varios elementas
	Escalabilidad	Aumento de hardware o servidores para distribuir la carga.	Mientras más nodos, mayor recursos en la red.	Escalable por capas independientes	Ilimitada al basarse en varias nodos
	Ejemplos reales	Aplicación de correo como Gmail.	Descarga de películas.	Sistemas bancarios, deliverys	Amazon
7	**Estructura**	Arquitectura donde un cliente pide o recibe datos a un servidor.	Arquitectura descentralizada con nodos que actúan como clientes y servidores.	Capas de tres a más, capa de presentación, negocio y datos	Servicios manejados por nodos
	Ventajas	Seguridad y fácil manejo de los usuarios.	Más nodos mayor eficiencia	Funcionalidades de cada capa, separadas	Basasmos nuestros servicios de manera individual
	Desventajas	Saturamiento por peticiones excesivas que provocan latencia.	Difícil administrar la red, el rendimiento puede variar	Se lo trabaja como monolito y si colapsa una capa, colapsa otra	Difulciltad al manejar diferentes servicios
	Escalabilidad	Por aumento de hardware.	Escalabilidad Horizontal	Intermedia, si una capa falla fallan las demás	Se puede interactuar y actualizar esos los cuales no son tan útiles en ciertos casos
	Ejemplos reales	Navegación Web.	BitTorrent	Amazon (interfaz, nube, data)	Spotify

6. Resultados esperados:

Al finalizar la práctica, el estudiante deberá haber obtenido y documentado lo siguiente:

- Documento de investigación de la arquitectura asignada (definición, componentes, ventajas/desventajas, escalabilidad y 2 ejemplos reales).
- Un diagrama digital de la arquitectura asignada (draw.io o similar).
- Matriz comparativa grupal completa (integrando las cuatro arquitecturas con los aportes de todos los equipos).
- Diseño del caso de estudio: diagrama + justificación técnica (mínimo 3 argumentos) + identificación de transparencia, escalabilidad y tolerancia a fallos.
- Reporte individual breve (máx. 1 página) con conclusiones sobre qué arquitectura considera más versátil y por qué.

7. Diseño del caso de estudio

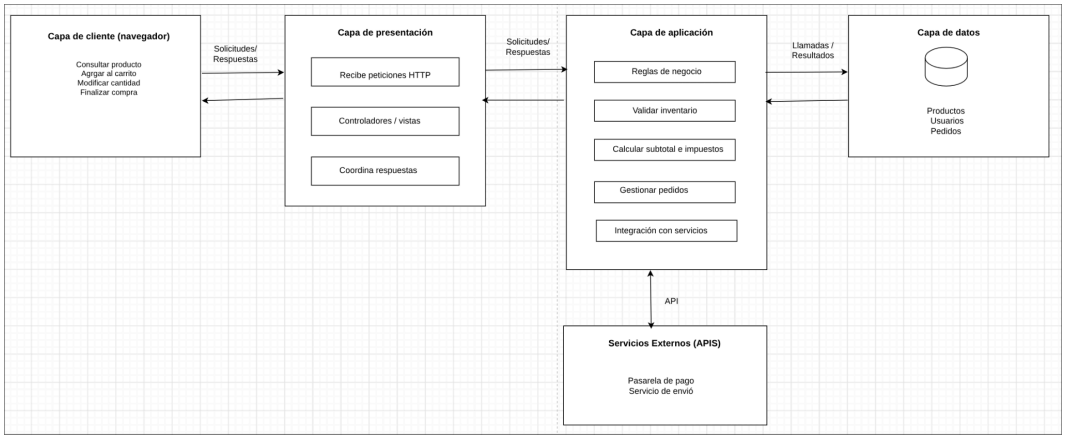


Figura 5: Diagrama del caso de estudio Multicapa

- **Justificación técnica:** Cada capa cumple con una función específica, la capa de presentación controla las solicitudes y evita el acceso directo del cliente a la base de datos. Por otra parte, la capa de aplicación conecta las funcionalidades del sistema con servicios externos, como pasarelas de pago y envíos.
- **Identificación de transparencia, escalabilidad y tolerancia a fallos:** En cuanto a la transparencia el usuario interactúa con el sistema sin conocer los procesos internos. La escalabilidad permite ampliar los recursos de cada capa según el aumento de usuarios y transacciones. En cuanto a la tolerancia a fallos la división de capas ayuda a separar errores, puede fortalecerse mediante respaldos, redundancia y recuperación ante fallos.

8. Conclusiones:

- Durante el desarrollo de la práctica se fortaleció la capacidad para identificar y comparar las características estructurales, ventajas y limitaciones de las arquitecturas como cliente-servidor, peer-to-peer, multicapa y microservicios.
- Se utilizó como estrategia práctica diseñar casos de estudio reales, justificando técnicamente la elección de una arquitectura y reconociendo conceptos clave como lo son la escalabilidad, la tolerancia a fallos y la transparencia.
- Consideramos que la arquitectura más versátil es Microservicios

9. Preguntas de Control

- **¿Qué es un sistema distribuido y cuáles son sus características fundamentales que lo diferencian de un sistema centralizado?**

  Un sistema distribuido son varias computadoras que se comunican mediante una red para compartir recursos, intercambiar información y ejecutar tareas de manera coordinadamente, funcionando como un único sistema. A diferencia de un sistema centralizado, donde el procesamiento y la administración de recursos dependen de un servidor central, un sistema distribuido divide las responsabilidades entre varios nodos. Sus características fundamentales son las siguientes [1]:

- **Concurrencia:** Varios nodos pueden ejecutar procesos simultáneamente y atender múltiples solicitudes
- **Escalabilidad:** Se puede aumentar la capacidad del sistema incorporando nuevos equipos o servidores.
- **Tolerancia a fallos:** Logra que el servicio no se corte si un servidor falla, usando un respaldo automático.
- **Compartición de recursos:** Permite distribuir y utilizar datos, almacenamiento y capacidad de procesamiento entre diferentes equipos.
- **Transparencia:** Busca que el usuario perciba el sistema como una única aplicación, sin necesidad de conocer su distribución interna.

- **Explique la diferencia de comunicación entre la arquitectura cliente-servidor y la arquitectura peer-to-peer.**

  La diferencia radica en cómo se establece la comunicación y se distribuyen las funciones entre los nodos, ya sea mediante un modelo centralizado o descentralizado. En la arquitectura cliente-servidor, la comunicación es centralizada, ya que los clientes realizan solicitudes a servidores que proporcionan los recursos o servicios. En cambio, en la arquitectura peer-to-peer, los nodos pueden actuar simultáneamente como clientes y servidores, permitiendo así que los recursos y servicios se compartan directamente entre ellos [2].

- **¿Qué rol cumple el middleware en un sistema distribuido y en qué capa se ubica?**

  El middleware cumple el rol de actuar como un intermediario entre las aplicaciones y los sistemas que forman parte de un sistema distribuido, facilitando así la comunicación y el intercambio de información entre ellos. Con respecto a su ubicación, este se encuentra en una capa

intermedia entre las aplicaciones y el sistema operativo o los recursos del sistema [2].

- **En la arquitectura de tres capas (3-tier), ¿qué función cumple cada capa y por qué separarlas mejora el mantenimiento?**

  En el caso de la capa de presentación, esta se encarga de la interfaz de usuario, capturando las entradas y mostrando los datos de manera gráfica e interactiva. Por su parte, la capa de negocio se encarga de procesar la lógica de la aplicación tanto aplicando reglas de negocio como gestionando el flujo de trabajo. Por último, la capa de datos se encarga de administrar el acceso, modificación y la persistencia de la información [3].

  En cuanto al porqué separar las capas mejora el mantenimiento, se debe al hecho de que al estar separadas estrictamente, se permite dividir las responsabilidades, además de permitir realizar modificaciones o actualizaciones en cada capa de forma totalmente independiente [3].

- **¿Cuáles son las principales ventajas de los microservicios frente a un diseño monolítico, y qué desafíos introducen?**

  La arquitectura de microservicios trae como ventaja el escalado independiente, es decir, se puede escalar solo aquellos servicios que experimenten cuellos de botella. Además, proporcionan un aislamiento a fallos, ya que un error en un microservicio no provoca la caída de toda la aplicación [2].

  En cuanto a los desafíos introducidos, se puede hablar de complejidad operativa, ya que se requiere la administración y orquestación de múltiples flujos de trabajo, ya sea coordinar la consistencia de los datos, considerar la latencia del intercambio de mensajes, o los cambios estructurales en las APIs [4].

- **¿Qué arquitectura consideraría más adecuada para una aplicación como BitTorrent y por qué?**

  Consideraría que una de las arquitecturas más adecuadas sería Peer-to-Peer, ya que BitTorrent está diseñado para la distribución masiva de archivos grandes sin depender de servidores centrales. Además, al permitir que los clientes actúen también como servidores, se logra un incremento en el ancho de banda y con ello, mayor disponibilidad y menor probabilidad de aparecer cuellos de botella [5].

- **¿Cómo influye la escalabilidad horizontal y vertical en la elección de una arquitectura distribuida?**

  La escalabilidad vertical consiste en aumentar la capacidad de un servidor mediante más recursos como CPU, memoria RAM o almacenamiento; mientras que la escalabilidad horizontal significa agregar nuevos servidores para distribuir la carga de trabajo.

  En una arquitectura distribuida la escalabilidad horizontal permite repartir las solicitudes entre múltiples nodos, mejorar la disponibilidad y reducir la dependencia de un único servidor. Por otra parte, la escalabilidad vertical puede facilitar el incremento de capacidad de un nodo existente, aunque está limitada por las características físicas y económicas del hardware.

La elección depende del volumen de usuarios, la carga de procesamiento, el presupuesto, la tolerancia a fallos y las necesidades de crecimiento del sistema [6].

- **¿Qué significa transparencia en un sistema distribuido y mencione dos tipos de transparencia?**

  La transparencia en un sistema distribuido oculta la complejidad operativa de los recursos para que múltiples computadoras funcionen como una única plataforma. Esta propiedad es muy importante en el diseño de sistemas distribuidos, pues permite abstraer el funcionamiento interno para los usuarios y las aplicaciones [7].

  1. **Transparencia de acceso:** Usa los recursos locales y remotos de la misma manera, ocultando las diferencias en los mecanismos de acceso y representación de los datos.
  2. **Transparencia de ubicación:** Accede a un recurso sin conocer su ubicación física dentro de la red, el sistema se encarga de localizarlo mediante nombres o identificadores lógicos.

- **Si un servidor central falla, ¿qué arquitectura continúa funcionando y por qué?**

  Sigue funcionando la arquitectura distribuida descentralizada, especialmente cuando implementa mecanismos de replicación y tolerancia a fallos, puede continuar funcionando ante la caída de un servidor central ya que sus servicios y recursos se encuentran distribuidos entre varios nodos [7].

- **En su caso de estudio, ¿qué criterios fueron decisivos para elegir la arquitectura propuesta?**

  Arquitectura multicapa porque permite la separación estricta de responsabilidades (presentación, negocio y datos) que facilita el mantenimiento, la seguridad centralizada y control del acceso a la información, y la escalabilidad independiente por capas para manejar grandes volúmenes de usuarios y datos.

10. Declaración de uso de IA

Para el desarrollo de la presente práctica se emplearon herramientas de Inteligencia Artificial con fines de apoyo académico y técnico. En particular, se utilizó NotebookLM como recurso para facilitar el entendimiento de la temática y la estructura conceptual de la práctica.

Prompts:

“Estructura, ventajas, desventajas y escalabilidad de (aqui escribimos la arquitectura correspondiente a cada punto).”

11. Bibliografía

[1] N. Naik, “Demystifying properties of distributed systems,” in 2021 IEEE International Symposium on Systems Engineering (ISSE), Vienna, Austria, 2021, pp. 1–8, doi: 10.1109/ISSE51541.2021.9582515.

[2] S. S. Gill, H. Wu, P. Patros, C. Ottaviani, P. Arora, V. C. Pujol, D. Haunschild, A. K. Parlikad, O. Cetinkaya, H. Lutfiyya, V. Stankovski, R. Li, Y.

Ding, J. Qadir, A. Abraham, S. K. Ghosh, H. H. Song, R. Sakellariou, O. Rana, J. J. P. C. Rodrigues, S. S. Kanhere, S. Dustdar, S. Uhlig, K. Ramamohanarao, and R. Buyya, "Modern computing: Vision and challenges," Telematics and Informatics Reports, vol. 13, Art. no. 100116, 2024, doi: 10.1016/j.teler.2024.100116.

[3] R. J. Celí Párraga, M. F. Boné Andrade, and A. P. Mora Olivero, Programación Web del Frontend al Backend. Esmeraldas, Ecuador: Editorial Grupo AEA, 2023. [Online]. Available: https://open.umn.edu/opentextbooks/textbooks/programacion-web-del-frontend-al-backend

[4] A. Lercher, "Managing API evolution in microservice architecture," in Proc. 2024 IEEE/ACM 46th Int. Conf. Software Engineering: Companion Proc. (ICSE-Companion), Lisbon, Portugal, 2024, pp. 195–197, doi: 10.1145/3639478.3639800.

[5] F. Dai, M. A. Hossain, and Y. Wang, "State of the art in parallel and distributed systems: Emerging trends and challenges," Electronics, vol. 14, no. 4, Art. no. 677, 2025, doi: 10.3390/electronics14040677.

[6] Blinowski, G., A. Ojdowska, and A. Przybyłek, "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," IEEE Access, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.

[7] "Demystifying Properties of Distributed Systems," in 2021 IEEE International Symposium on Systems Engineering (ISSE), 2021, doi: 10.1109/ISSE51541.2021.9582515